

Mitigasi Serangan Replay pada API Stateless Menggunakan Derivasi Salt Deterministik Berbasis Korpus Teks Publik dalam Arsitektur Serverless

Atharizza Muhammad Athaya - 18223079

Program Studi Sistem dan Teknologi Informasi

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung, Jalan Ganesha 10 Bandung

E-mail: atharizza535@gmail.com , 18223079@std.stei.itb.ac.id

Abstrak—*Replay attack* tetap menjadi permasalahan pada autentikasi API karena *request* yang valid sebelumnya dapat ditangkap dan dikirim ulang. Makalah ini mendesain dan mengevaluasi mekanisme *salt* deterministik dari korpus publik untuk pengikatan konteks pada API *stateless*. *Salt* diturunkan dari *canonical request*, identitas klien, *timestamp*, *time bucket*, versi korpus, dan *registry hash*, lalu dimasukkan ke dalam *signing payload* yang diautentikasi menggunakan HMAC-SHA256. Korpus publik tidak diposisikan sebagai rahasia atau sumber entropi, tetapi sebagai bahan konteks publik yang dapat direproduksi dan diaudit oleh klien maupun verifikasi. Prototipe yang diajukan diimplementasikan menggunakan TypeScript dan Hono.js. Hasil pengujian menunjukkan bahwa *request* valid diterima, sementara *request* dengan *body*, *query*, *path*, identitas klien, versi korpus, *registry hash*, atau *timestamp* yang tidak sesuai ditolak. Prototipe juga mengekspos metadata verifikasi, seperti *selectedCorpusId*, *fragmentOffset*, *publicCorpusSalt*, *registryHash*, *timeBucket*, dan alasan penolakan, sehingga proses pengikatan konteks dapat diamati secara eksplisit. Walaupun begitu, verifikasi *stateless* tetap dapat menerima *replay* identik di dalam rentang waktu yang sama karena tidak menyimpan *replay memory*. Oleh karena itu, mekanisme ini diposisikan sebagai *layer* pengikat konteks *stateless* yang mempersempit *replay* lintas waktu, *endpoint*, klien, *body*, dan versi korpus, bukan sebagai pengganti *nonce store*, *replay cache*, atau mekanisme *idempotency* pada *endpoint* kritis.

Kata Kunci—API security, replay attack, HMAC, serverless, verifikasi stateless, korpus publik, pengikatan konteks.

I. PENDAHULUAN

Replay attack adalah permasalahan yang sudah ada sedari lama dalam protokol autentikasi. Pada serangan ini, penyerang akan menangkap pesan atau *request* yang valid sebelumnya dan mengirim ulang pesan tersebut untuk mendapatkan manfaat yang sama, atau melakukan impersonasi terhadap pengirim awal yang sah. Pada sistem API, permasalahan ini akan menjadi signifikan ketika sebuah *request* memicu permintaan yang bersifat sensitif, seperti perubahan atau modifikasi data, pembuatan token, konfirmasi transaksi, atau akses terhadap sumber daya yang dilindungi. Studi protokol autentikasi yang telah dilakukan oleh Needham dan Schroeder, atau Denning dan Sacco menunjukkan bahwa penggunaan ulang pesan telah menjadi perhatian utama dalam autentikasi jaringan dalam beberapa dekade terakhir [1], [2]. National

Institute of Standards and Technology (NIST) juga menempatkan keamanan terhadap serangan ini menjadi hal yang penting pada mekanisme autentikasi yang diterapkan, terutama ketika penyerang dapat menangkap dan menggunakan kembali pesan yang valid [3].

Permasalahan ini akan menjadi lebih rumit pada arsitektur API *stateless*. Secara konsep, HTTP didefinisikan sebagai protokol *request/response stateless* yang berjalan di atas layer protokol aplikasi, menunjukkan bahwa setiap *request* yang dikirim akan diinterpretasikan secara independen selama aplikasi tidak menambahkan *layer* manajemen *state*-nya sendiri [4]. Pencegahan serangan *replay* yang kuat biasanya bergantung pada penyimpanan *request* valid sebelumnya, seperti penggunaan *nonce*, *idempotency key*, sidik jari (*fingerprint*), atau *replay memory*.

Makalah ini mengajukan desain mitigasi serangan *replay* untuk API *stateless* dengan menggunakan *salt* deterministik yang diturunkan dari korpus publik. Prototipe ini menggunakan korpus publik sebagai penyedia materi konteks yang diturunkan dari konteks *request* yang diberikan, *time bucket*, versi korpus, dan *hash registry* yang kemudian ditandatangani dengan *payload* HMAC-SHA256.

Terdapat empat kontribusi yang dibawakan makalah ini, mencakup: 1) makalah ini merumuskan prototipe penandatanganan *request* HMAC *stateless* yang menggabungkan cakupan *request* kanonik, *time bucket*, versi korpus, *registry hash*, dan *salt*; 2) makalah ini memosisikan korpus publik yang digunakan sebagai material yang dapat diaudit, bukan sebagai penyedia rahasia atau entropi acak; 3) makalah ini mengimplementasikan prototipe TypeScript/Hono yang mengekspos metadata penandatanganan, derivasi, verifikasi, dan status penolakan; dan 4) makalah ini mengevaluasi batasan prototipe terhadap serangan *replay*, terutama *replay* dalam rentang waktu yang sama yang tidak dapat dibedakan tanpa memori tambahan pada arsitektur *stateless*.

II. DASAR TEORI

A. Replay Resistance dan Freshness

Replay resistance berkaitan dengan kemampuan protokol untuk menolak penggunaan ulang pesan yang sebelumnya

valid. Dalam literatur autentikasi, konsep *freshness* menjelaskan bahwa pesan yang valid secara kriptografis masih bersifat rentan ketika dapat digunakan kembali di luar konteks atau waktu yang dimaksudkan [1], [2].

B. Penandatanganan Request HTTP dan Canonicalization

Penandatanganan *request* HTTP akan mengikat komponen-komponennya kepada tanda tangan terkait, mencakup metode HTTP, *path*, *query*, *header*, *timestamp*, dan *body digest*. RFC 9421 mendefinisikan sebuah *framework* untuk HTTP *Message Signatures* dan memfokuskan perlunya memilih dan merepresentasikan komponen HTTP secara konsisten sebelum dilakukannya penandatanganan dan verifikasi [5]. Hal ini penting karena tanda tangan hanya melindungi komponen yang memang sudah ditandatangani.

Oleh karena itu, *canonicalization* adalah bagian dari *security surface*. Ketika klien dan server merepresentasikan *path*, *query*, *header*, atau *body* secara berbeda, maka *request* valid dapat ditolak.

C. HMAC sebagai Mekanisme Autentikasi

HMAC memberikan autentikasi dan integritas pesan menggunakan kunci rahasia bersama [6], [7]. Makalah ini menggunakan HMAC-SHA256 sebagai mekanisme autentikasi utama. *Request* hanya dapat diterima ketika verifikasi dapat merekonstruksi *payload* tanda tangan yang sama sekaligus memvalidasi HMAC menggunakan kunci klien. *Salt* yang diturunkan oleh korpus publik tidak menggantikan kunci HMAC, tidak menjadi rahasia kedua, atau menambah keamanan lewat kerahasiaan. Jika rahasia HMAC dibocorkan, maka korpus publik sendiri tidak dapat mencegah *request* palsu, sebagaimana mekanisme penandatanganan menggunakan kunci bersama lainnya.

D. HTTP Stateless, Idempotency, dan Replay Memory

HTTP didefinisikan sebagai protokol *request/response stateless* yang berjalan di atas *layer* aplikasi [4]. Hal ini berarti setiap *request* akan diinterpretasikan secara independen selama tidak ada penambahan mekanisme manajemen *state* pada aplikasi terkait. Sifat *stateless* ini berguna untuk skalabilitas dan *deployment*, namun juga berarti verifikasi tidak dapat mengetahui secara langsung apakah sebuah *request* sudah diproses.

Pertahanan *replay* yang bersifat *stateful* menyediakan deteksi duplikasi yang lebih kuat. Dalam hal ini, penyimpanan *nonce*, *replay cache*, *fingerprint*, atau mekanisme *idempotency* memungkinkan server untuk menolak *request* berulang. Draft *header* HTTP Idempotency-Key mendeskripsikan mekanisme klien dalam menyisipkan kunci unik kepada *request* supaya server dapat menemukan pengulangan submisi [8]. Mekanisme ini akan berguna pada operasi non-idempotent, namun membutuhkan *state* dari sisi server, kebijakan ekspirasi, dan sinkronisasi kuat pada *deployment* terdistribusi.

E. Korpus Publik sebagai Konteks Deterministik

Pada makalah ini, *salt* yang dimaksudkan mengacu kepada materi publik deterministik yang diturunkan dari korpus yang terikat pada versi tertentu. *Salt* tidak berfungsi sebagai rahasia tambahan, sumber entropi acak, atau menggantikan kunci bersama HMAC pada mekanisme autentikasi.

Desain yang diajukan menggunakan korpus publik karena materi berikut dapat dinormalisasi, di-hash, dikutip, dan diaudit secara independen. Versi yang statik dapat mengidentifikasi konfigurasi yang digunakan, dan *registry hash* dapat mendeteksi perbedaan konfigurasi tersebut. *Snapshot* korpus menambahkan referensi eksternal yang dapat direproduksi, di mana *offset* fragmen yang dipilih dapat bervariasi sesuai konteks *request* yang diberikan, namun tetap dapat direkomputasi oleh pihak klien dan verifikasi.

Korpus yang digunakan tidak membuat tanda tangan lebih sulit ditebak dengan menyembunyikannya. Penyerang diasumsikan mengetahui korpus yang digunakan, format *registry*, algoritma derivasi, dan *header* publik yang disediakan. Adapun fungsi korpus terdapat pada pengikatan versi, *provenance*, reproduisibilitas, dan pengikatan konteks publik yang bergantung pada *request*.

F. Gap Penelitian

Mekanisme yang ada sudah menyediakan autentikasi, kesegaran, dan ketahanan terhadap serangan *replay* dalam bentuk yang beragam. Penyimpanan seperti *nonce store*, *replay cache*, dan catatan *idempotency* lebih kuat dalam mendeteksi *request* duplikat. Hal ini disebabkan oleh kemampuannya dalam mengingat submisi sebelumnya. HMAC yang diberikan *timestamp* lebih mudah dijalankan pada arsitektur *stateless*, namun hanya dapat membatasi masa valid *request* dan tidak dapat mendeteksi penggunaan ulang yang dilakukan pada rentang waktu yang sama.

Karenanya, ruang desain yang tersisa cenderung terbatas, mencakup pengayaan ikatan *request* secara deterministik dan dapat diaudit tanpa mengubah verifikasi menjadi lingkungan *stateful* yang memiliki memori. Makalah ini mengeksplorasi ruang tersebut dengan menurunkan *salt* dari materi korpus publik yang diikat oleh versi tertentu dan konteks *request* yang telah diautentikasi.

III. DEFINISI MASALAH DAN TUJUAN DESAIN

A. Definisi Masalah

Adapun permasalahan yang diangkat pada makalah ini adalah sebagai berikut:

Bagaimana ikatan sebuah request pada API stateless dapat diperkuat menggunakan salt deterministik dari korpus publik sembari menjaga reproduisibilitas tanpa menerapkan replay memory untuk setiap request?

B. Tujuan Desain

Adapun desain yang diajukan makalah ini memiliki empat tujuan utama, mencakup:

- 1) Verifikasi *stateless*: Server dapat memverifikasi *request* tanpa menyimpan *request*, *nonce*, atau tanda tangan *request* yang telah diterima sebelumnya.
- 2) *Context-based binding*: Adapun tanda tangan HMAC yang dibuat harus mengikat metode HTTP, *path*, *query* kanonik, *body*, identitas klien, *timestamp*, dan *bucket* waktu. Setiap perubahan konteks *request* mengubah *payload* penandatanganan.
- 3) *Corpus-salt binding*: *Request* yang telah ditandatangani harus mencakup metadata *salt* korpus

publik yang diturunkan dari *registry* korpus publik yang telah diberi versi. Korpus berfungsi sebagai penyedia konteks yang dapat direproduksi dan bukan sebagai penyedia rahasia autentikasi.

- 4) Reprodusibilitas deterministik: Server dan klien harus dapat menurunkan entri korpus, fragmen, *salt* korpus, dan *registry hash* yang konsisten, jika diberikan konteks request, versi korpus, dan *bucket* identik.

C. Model Ancaman dan Asumsi Kepercayaan

Penyerang diasumsikan dapat mengamati dan meniru *request* API yang telah valid sebelumnya. Penyerang dapat mengirim kembali *request* tanpa dilakukannya perubahan, atau melakukan modifikasi metode, *path*, *query*, *body*, identitas klien, *timestamp*, versi korpus, *registry hash*, konteks derivasi *salt*, atau tanda tangan HMAC. Penyerang juga diasumsikan memiliki akses terhadap korpus publik, versi korpus, mekanisme kanonik, algoritma derivasi *salt*, versi protokol, dan *time bucket* yang diterima.

Penyerang diasumsikan untuk tidak mengetahui rahasia HMAC milik klien yang sah. Jika rahasia HMAC diketahui, maka mekanisme penandatanganan *request* akan gagal seperti skema berbasis HMAC lainnya, karena penyerang dapat membuat tanda tangan yang valid untuk *payload* yang dikirimkan. Korpus publik tidak dapat memitigasi kebocoran.

Adapun *trust model* direpresentasikan pada tabel ini:

TABEL I. TRUST MODEL

Komponen	Asumsi
Rahasia klien HMAC	Dipegang secara aman oleh klien sah
<i>Registry</i> rahasia klien	Dipetakan dengan id klien.
Korpus publik	Publik, memiliki versi, dan menggunakan <i>snapshot</i> konsisten antara klien dan server
Algoritma derivasi <i>salt</i>	Publik dan deterministik
Aturan kanonik	Identik antara penandatanganan dan verifikator
Penyerang jaringan	Dapat mengamati, meniru, mengubah, dan melakukan serangan <i>replay</i> request
<i>Replay memory</i>	Tidak tersedia pada mode verifikasi <i>stateless</i> ; dapat ditambahkan sebagai kontrol terpisah

IV. METODE YANG DIAJUKAN

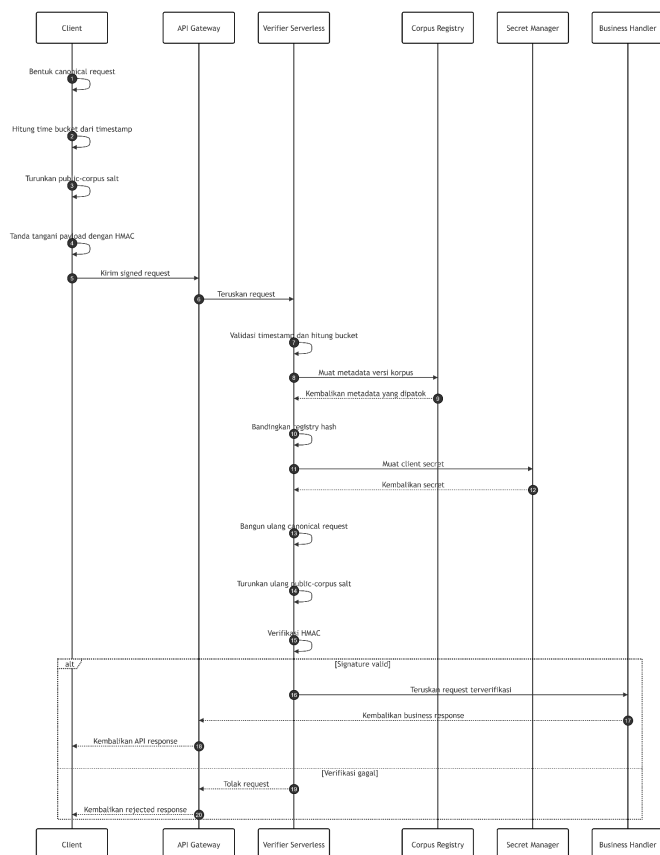
A. Model Signature Baseline

Model penandatanganan dasar menggunakan *request* HTTP kanonik dan HMAC-SHA256. *Request* mencakup komponen metode HTTP, *path*, *query*, *header*, *body hash*, ID klien, *timestamp*, *time bucket* yang telah dikomputasi, versi korpus, dan *registry hash*. Klien kemudian menghitung HMAC di atas *payload* penandatanganan menggunakan rahasia bersama. Verifikator merekonstruksi *payload* yang sama dan membandingkannya dengan HMAC yang diharapkan pada rentang waktu yang konstan.

Proses dasar ini mengikuti alur praktik penandatanganan *request* yang sudah diterapkan, dengan tujuan tanda tangan dapat mengautentikasi secara tepat komponen apa saja yang dicakup pada representasi *request* kanonik. Hal ini sesuai dengan kebutuhan representasi komponen HTTP yang stabil pada RFC 9421 [5], sekaligus selaras secara struktural pada praktik *request* kanonik di AWS Signature Version 4 [9] dan tanda tangan digital OAuth 1.0 [10]. Kontribusi makalah ini adalah penambahan materi konteks publik dari *registry* korpus yang dapat direproduksi secara deterministik dan terikat versi.

Gambar 1 menunjukkan alur penandatanganan dan verifikasi yang diterapkan pada prototipe yang diajukan.

Gambar 1. Alur penandatanganan dan verifikasi



B. Derivasi Salt Berbasis Korpus Publik

Salt dari korpus publik diturunkan secara deterministik dari *request* kanonik dan daftar entri korpus. Prototipe menghitung sebuah *seed* dari *request* yang diberikan, yang kemudian akan digunakan untuk memilih satu entri korpus dari daftar yang disediakan, dan memilih *offset* fragmen dari *snapshot* korpus tersebut. Fragmen terpilih kemudian dibaca dengan panjang yang tetap dan secara deterministik akan diikat di akhir korpus, jika dibutuhkan. Fragmen tersebut kemudian di-hash dengan metadata korpus untuk menghasilkan variabel *publicCorpusSalt*.

Salt yang dihasilkan kemudian ditambahkan kepada *request* kanonik sebelum dilakukannya perhitungan HMAC. Hal ini akan memengaruhi perubahan pada *body*, *endpoint*,

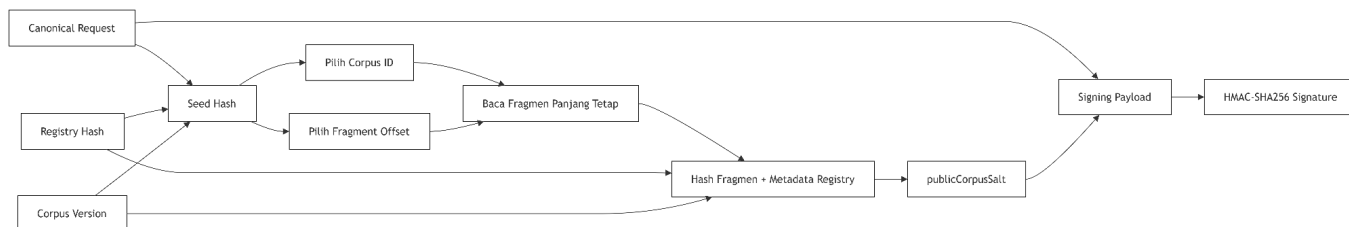
query, ID klien, *timestamp*, *time bucket*, versi korpus, atau *registry hash* mengubah *payload* penandatanganan untuk berubah. Karena HMAC mengautentikasi *payload* terakhir, maka verifikator akan menolak konteks yang diubah kecuali ketika penyerang bisa menghitung HMAC yang valid.

Teks korpus yang berulang atau dapat diprediksi tidak mengurangi kerahasiaan, karena korpus tidak digunakan

sebagai rahasia sedari awal. Adapun komponen penting pada prototipe ini adalah bagaimana sifatnya yang deterministik memiliki ID korpus, *offset* fragmen, dan *salt* yang sama pada implementasi klien dan verifikator.

Gambar 2 di bawah ini membatasi fase derivasi *salt* antara kanonisasi dan komputasi HMAC.

Gambar 2. Alur derivasi *salt* korpus publik



C. Registry Korpus Terikat Versi

Registry korpus berfungsi sebagai konfigurasi verifikasi tetap. Pada prototipe ini, *registry* menggunakan versi 2026-06-real-v1, normalisasi utf8-nfc-lf-trim, dan panjang fragmen 512 byte. Setiap entri memuat metadata sumber, *path* snapshot, lisensi, dan *hash* ternormalisasi.

Tabel II menunjukkan *snapshot* korpus yang telah diberikan versi dan digunakan oleh prototipe.

TABEL II. REGISTRY KORPUS TERIKAT VERSI

ID	Sumber	Hash
gutenberg-alice-11	Project Gutenberg ebook #11, Alice's Adventures in Wonderland	sha256:2f06af7f...07a3c1c
gutenberg-world-english-bible-8294	Project Gutenberg ebook #8294, The World English Bible	sha256:ae52d7d7...387cc8
tanzil-quran-simple-clean-v1.1	Tanzil Quran Simple Clean text, Version 1.1	sha256:ea7947fd...3fa8b1

Registry hash kemudian dikirimkan dengan *request* sebagai variabel X-Registry-Hash dan dihitung kembali oleh verifikator. Perbedaan hasil menunjukkan bahwa terdapat perbedaan konfigurasi korpus yang digunakan antara penandatanganan dan verifikator. Verifikator kemudian menolak *request* tersebut sebelum menerima operasi selanjutnya.

D. Status Verifikasi

Verifikator membaca ID klien, *timestamp*, versi korpus, *registry hash*, daftar *header*, dan tanda tangan dari *request*. *Request* yang tidak lengkap atau telah diubah ditolak sebelum perbandingan HMAC, dengan alasan penolakan yang ditunjukkan pada Tabel III.

TABEL III. STATUS KEGAGALAN

Status	Deskripsi
missing_client_id	Identitas klien kosong

missing_timestamp	<i>Timestamp</i> kosong atau bukan <i>integer</i> valid
missing_corpus_version	Versi korpus kosong
missing_registry_hash	<i>Registry hash</i> kosong
missing_signature	Tanda tangan HMAC kosong
unknown_client	ID klien tidak diketahui
unknown_corpus_version	Versi korpus tidak cocok dengan verifikator
registry_hash_mismatch	<i>Registry hash</i> klien dan verifikator berbeda
timestamp_expired	<i>Request</i> berada di luar toleransi kesegaran
signature_mismatch	HMAC hasil rekalkulasi tidak cocok
invalid_request	<i>Request</i> tidak dapat dikanonisasi atau diverifikasi secara aman

Dari *request* yang valid secara struktural, verifikator akan menghitung *time bucket* dari *timestamp*, menganonikasi ulang *request*, menghitung kembali *registry hash*, menurunkan *salt* korpus publik yang sama, merekonstruksi *payload* penandatanganan, dan memeriksa HMAC. *Request* yang diubah akan gagal karena *payload* penandatanganan yang dihitung kembali akan berbeda dengan yang digunakan klien.

V. IMPLEMENTASI PROTOTIPE

Prototipe dirancang menggunakan *tech stack* TypeScript dan Hono.js untuk memantau desain yang diajukan. Adapun prototipe yang dibuat tidak dirancang sebagai kontrak API level produksi, atau tutorial bagi pengguna. Prototipe hanya dirancang untuk memudahkan pemeriksaan penandatanganan, derivasi, verifikasi, dan penolakan.

Prototipe mencakup fungsi modular untuk kanonisasi, validasi entri korpus, derivasi *salt*, penandatanganan HMAC, verifikasi, *middleware* bersifat *serverless*, *endpoint* lab serangan *replay*, dan penambahan kemampuan *idempotency*. Perilaku prototipe diekspose pada metadata *debug* untuk

mendukung klaim evaluasi. Adapun pemetaan tujuan evaluasi dan metadata prototipe ditunjukkan pada Tabel IV.

TABEL IV. PENGGUNAAN METADATA DAN EVALUASI PROTOTIPE

Metadata	Fungsi
canonicalRequestHash	Mengonfirmasi kanonikasi deterministik
bodyHash	Mendukung analisis pengubahan <i>body</i>
selectedCorpusId	Menunjukkan pemilihan korpus
fragmentOffset	Menunjukkan derivasi <i>offset</i>
publicCorpusSalt	Menunjukkan materi publik yang direproduksi
corpusVersion	Mengonfirmasi ikatan versi korpus
registryHash	Mengonfirmasi ikatan <i>registry hash</i>
timeBucket	Mendukung analisis <i>timestamp</i> dan <i>bucket</i>
verificationStatus / rejectionReason	Mengklasifikasikan request yang diterima dan ditolak
idempotencyStatus	Membedakan verifikasi <i>stateful</i> dan <i>stateless</i>

VI. RANCANGAN EVALUASI

Evaluasi diawali dengan menguji kebenaran *baseline: request* valid harus diterima, sedangkan masukan identik harus menghasilkan *request hash*, korpus terpilih, *offset* fragmen, salt, dan HMAC yang sama.

Kedua, kekuatan pengikatan menanyakan apakah perubahan yang tidak disengaja dapat mengubah *request* dan merusak verifikasi. Perubahan yang dimaksud mencakup isi *body*, *query*, *path*, *endpoint*, identitas klien, versi korpus, dan *registry hash*. Jika terdapat perubahan pada komponen-komponen berikut, maka desain harus menghasilkan penolakan, karena verifikator merekonstruksi *payload* penandatanganan yang berbeda.

Terakhir, analisis batasan *replay* menanyakan di mana desain *stateless* yang diajukan terhenti. Serangan *replay* yang kedaluwarsa sudah seharusnya gagal karena *timestamp* berada di luar rentang *clock skew* yang diterima. Serangan pada rentang waktu yang sama akan tetap sukses pada mode *stateless* karena verifikator tidak memiliki informasi terkait penggunaan sebelumnya. Ketika kemampuan *idempotency* diberikan, maka submisi duplikat dapat ditolak karena verifikator kini memiliki *state* pencatat.

Tabel V meringkas skenario evaluasi yang dilakukan dan keluaran yang diharapkan di setiap skenario.

TABEL V. SKENARIO EVALUASI

Skenario	Keluaran yang Diharapkan
<i>Signed request</i> valid	Diterima; metadata deterministik stabil
<i>Body tampering</i>	Ditolak dengan <i>signature mismatch</i>

<i>Query</i> diubah	Ditolak dengan <i>signature mismatch</i>
Antar <i>endpoint</i>	Ditolak dengan <i>signature mismatch</i>
<i>Cross-client replay</i>	Ditolak karena identitas klien mengubah penandatanganan <i>payload</i>
<i>Unknown corpus version</i>	Ditolak sebelum <i>business handling</i>
<i>Registry hash mismatch</i>	Ditolak sebelum <i>business handling</i>
<i>Timestamp</i> kedaluwarsa	Ditolak karena <i>timestamp</i> di luar toleransi
<i>Same-window replay, stateless mode</i>	Dapat diterima; menunjukkan batas <i>stateless</i>
<i>Same-window replay, idempotency mode</i>	Ditolak sebagai duplikat (<i>stateful</i>)

VII. HASIL DAN ANALISIS

Otomasi tes yang dilakukan mendukung klaim perilaku yang diharapkan, di mana *request* yang diharapkan valid diterima oleh verifikator. *Request* dengan tanda tangan yang tidak ada, klien yang tidak diketahui, *timestamp* yang kedaluwarsa, versi korpus yang tidak tercatat, *registry hash* yang berbeda, isi *body*, *path*, atau *query* yang diubah, dan serangan *replay* dengan klien yang berbeda ditolak secara keseluruhan. Vektor pengujian menetapkan keluaran deterministik untuk *registry hash*, *hash body*, korpus yang dipilih, *offset* fragmen, salt, *hash payload*, dan tanda tangan.

Pengujian serangan *replay* secara sengaja membiarkan limitasi utama pada lingkungan *stateless*. Serangan pada rentang waktu yang sama tetap diterima pada verifikator *stateless* karena *request* identik di dalam *time bucket* yang sama akan menghasilkan *request* kanonik, salt, dan tanda tangan HMAC yang sama. Hasil ini tidak disebabkan oleh kelemahan implementasi, melainkan mengikuti batasan lingkungan yang tidak memiliki memori.

Pengujian dengan modul lab menunjukkan perbedaan di antara verifikasi *stateless* dan deteksi pada lingkungan *stateful*. Pada mode *stateless*, serangan *replay* yang dilakukan secara langsung akan diterima ketika semua masukan yang telah ditandatangani tetap valid. Pada mode *idempotency*, submisi pertama yang masuk akan disimpan dalam bentuk tanda tangan, yang menyebabkan submisi identik kedua ditolak sebagai duplikat. Hal ini menunjukkan perbedaan fungsi yang kontras antara desain yang diajukan dengan tindakan preventif terhadap submisi duplikat yang membutuhkan *state* tambahan pada pengimplementasiannya. Hasil evaluasi dirangkum pada Tabel VI di bawah.

TABEL VI. RINGKASAN HASIL

Cakupan	Hasil	Interpretasi
<i>Baseline correctness</i>	<i>Request</i> valid diterima dan keluaran deterministik stabil	Penandatanganan dan verifikasi dapat direproduksi
<i>Binding</i>	<i>Body</i> , <i>query</i> , <i>path</i> , klien,	Tanda tangan terikat pada

<i>strength</i>	versi korpus, atau <i>registry hash</i> yang berubah ditolak	konteks <i>request</i> dan konfigurasi korpus
<i>Replay boundary</i>	Replay yang kedaluwarsa ditolak; <i>replay</i> dapat tetap valid pada mode <i>stateless</i> dalam rentang yang sama	Batas <i>replay</i> melekat pada verifikasi <i>stateless</i> yang tidak memiliki memori
<i>Stateful supplement</i>	Mode <i>idempotency</i> menolak submisi duplikat	Deteksi duplikat yang kuat membutuhkan <i>state</i>

VIII. DISKUSI

Jika dibandingkan dengan mekanisme *timestamped* HMAC dasar, desain yang diajukan menambahkan pengikatan terhadap versi korpus, pengikatan *registry hash*, dan materi konteks publik yang dependen bergantung kepada *request* yang diajukan. Penambahan berikut menyebabkan alur verifikasi lebih eksplisit dan dapat diaudit. *Request* juga tidak hanya terikat dengan konteks HTTP atau *timestamp*, namun juga entri korpus yang metadata dan hasil normalisasi *hash*-nya dapat diperiksa.

Desain yang diajukan akan cenderung lebih lemah secara desain dalam mendeteksi duplikat, jika dibandingkan dengan penyimpanan *nonce*, *idempotency* atau *replay cache*. Kelemahan yang dibawa oleh desain ini disengaja dan dinyatakan secara terbuka. Verifikator yang menolak untuk mengingat *request* sebelumnya tidak dapat secara andal membedakan penggunaan pertama dengan penggunaan berulang jika setiap komponen masukan identik dan masih berlaku pada rentang waktu yang diterima.

Posisi *salt* korpus publik yang dibawa makalah ini bukan sebagai pengganti dari *replay memory*, namun sebagai *layer stateless* yang menyempitkan ruang serangan *replay* dari segala batasan konteks. Desain ini membantu menolak *request* yang telah diubah, serangan antar *endpoint*, serangan antara klien, serangan yang kedaluwarsa, dan serangan dengan entri korpus dengan versi yang berbeda. Untuk *endpoint* dengan risiko tinggi, maka desain ini harus diterapkan bersama

dengan mekanisme *idempotency*, *nonce cache*, atau mekanisme validasi bisnis lainnya.

Desain yang diajukan juga bergantung penuh kepada aturan kanonikasi, normalisasi korpus, distribusi *registry*, toleransi *clock skew*, dan manajemen rahasia yang digunakan guna menjaga keamanan sistem. Perbedaan di antara klien dan kanonikasi verifikator dapat menyebabkan penolakan yang salah, atau ketidakjelasan verifikasi. *Registry* yang tidak berlaku dapat menyebabkan penolakan yang tidak penting. Rentang waktu dengan *clock skew* yang luas menyebabkan celah serangan bertambah. Permasalahan-permasalahan yang disebutkan tidak dapat dianggap sepele, karena merupakan bagian dari properti keamanan sistem.

IX. KESIMPULAN

Makalah ini mengajukan desain mitigasi terhadap serangan *replay* pada API *stateless* pada arsitektur *serverless* menggunakan pengikatan konteks dengan korpus publik yang telah diberikan versi tertentu. Desain yang diajukan menurunkan *salt* korpus publik dari *request* kanonik dan entri korpus, kemudian menggabungkannya dengan *payload* penandatanganan HMAC-SHA256. Korpus yang digunakan berbasis publik dan deterministik, dan autentikasi yang digunakan tetap bergantung kepada rahasia bersama HMAC.

Pendekatan ini memperkuat pengikatan *request* terhadap *body*, *endpoint*, klien, waktu, versi korpus, dan konfigurasi *registry*. Metadata verifikasi juga membantu menjelaskan alasan *request* diterima atau ditolak. Namun, pada lingkungan *stateless*, *replay* dalam rentang waktu yang sama tetap dapat diterima karena verifikator tidak menyimpan memori tambahan.

Dengan demikian, desain ini lebih tepat diposisikan sebagai *layer* mitigasi untuk penandatanganan *request stateless* dan *serverless*, bukan sebagai pencegahan *replay* secara penuh. *Endpoint* yang membutuhkan pencegahan duplikasi tetap memerlukan kontrol *stateful*, seperti *nonce*, *idempotency*, atau validasi bisnis lainnya.

REPOSITORY

Kode sumber, dokumentasi, dan hasil eksperimen dapat diakses melalui repository berikut:

<https://r.atharizza.com/gh-infosec>

REFERENSI

- [1] R. M. Needham and M. D. Schroeder, "Using encryption for authentication in large networks of computers," Communications of the ACM, vol. 21, no. 12, pp. 993–999, Dec. 1978, doi: 10.1145/359657.359659.
- [2] D. E. Denning and G. M. Sacco, "Timestamps in key distribution protocols," Communications of the ACM, vol. 24, no. 8, pp. 533–536, Aug. 1981, doi: 10.1145/358722.358740.
- [3] D. Temoshok et al., Digital Identity Guidelines: Authentication and Authenticator Management, NIST Special Publication 800-63B-4, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2025. doi: 10.6028/NIST.SP.800-63B-4.
- [4] R. Fielding, M. Nottingham, and J. Reschke, HTTP Semantics, RFC 9110, Internet Engineering Task Force, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>

- [5] A. Backman, J. Richer, and M. Sporny, HTTP Message Signatures, RFC 9421, Internet Engineering Task Force, Feb. 2024. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9421>
- [6] H. Krawczyk, M. Bellare, and R. Canetti, HMAC: Keyed-Hashing for Message Authentication, RFC 2104, Internet Engineering Task Force, Feb. 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2104>
- [7] National Institute of Standards and Technology, The Keyed-Hash Message Authentication Code (HMAC), Federal Information Processing Standards Publication 198-1, Jul. 2008. doi: 10.6028/NIST.FIPS.198-1
- [8] IETF HTTPAPI Working Group, The Idempotency-Key HTTP Header Field, Internet-Draft draft-ietf-httpapi-idempotency-key-header. [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-httpapi-idempotency-key-header>
- [9] Amazon Web Services, "Authenticating requests: AWS Signature Version 4," Amazon Simple Storage Service API Reference. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/API/sig-v4-authenticating-requests.html>
- [10] E. Hammer-Lahav, The OAuth 1.0 Protocol, RFC 5849, Internet Engineering Task Force, Apr. 2010. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5849>